HAMM-LIPPSTADT
UNIVERSITY OF APPLIED SCIENCES

FPGA based implementation of an IoT network

Department Lippstadt 2

Project work report

submitted by
Andrew Antwan Mikheal Fahmy Zakhary

Electronic Engineering
Mat.Nr.: 2210009
<andrew-antwan-mikheal-fahmy.zakhary@stud.hshl.de>

March 16, 2025

First supervisor: Prof. Dr. -Ing. Ali Hayek

Abstract

The integration of Field-Programmable Gate Arrays (FPGAs) into Internet of Things (IoT) networks allows the opportunity to use their flexible design and processing efficiency in distributed systems. This project aims to explore the feasibility of using a Nexys A7-100T FPGA board as a node in an Internet of Things (IoT) network, using a Raspberry Pi Pico W as a wireless node to communicate wirelessly to a Raspberry Pi. As this project uses multiple protocols as SPI, I2C, and MQTT, it demonstrates an implementation of a heterogeneous network built around an FPGA. The results highlight the advantages of using an FPGA for sensor data acquisition while utilizing a microcontroller for wireless transmission and an SBC as a broker, showcasing a practical approach to industrial automation and embedded systems.

Contents

1 Introduction **1**

2 Fundamentals **2**
2.1 Overview 2
2.2 Internet of Things (IoT) 2
2.3 MQTT 3
2.4 FPGA 4
2.5 Raspberry Pi Pico W 4
2.6 Raspberry Pi 4 5
2.7 Nexys A7-100T 6

3 Methodology **7**

4 Evaluation **9**
4.1 Overview 9
4.2 FPGA Design 9
4.2.1 Clock Divider 9
4.2.2 Seven Segment Display 10
4.2.3 I2C Master 11
4.2.4 SPI Slave 13
4.3 Pico W Design 15
4.3.1 Setting up SPI interface 15
4.3.2 MQTT setup 17
4.3.3 Main file 17
4.4 Raspberry Pi Design 18
4.5 Results 18

5 Summary **20**
5.1 Conclusion 20
5.2 Future plans 21

References **22**

1 Introduction

Industrial revolutions can be categorized into 4 distinct revolutions. The first of which is the industrial revolution brought around by the steam engine. The second revolution is the electrification brought around by batteries and transformers. The third one is the automation which was brought around by the invention of transistors and then further developments of computers and software. The fourth and final one so far, is brought around by the development of machine communication through networks known also as Internet of Things (IoT). IoT has revolutionized the way devices are connected together. As this technology enabled the connection of huge amounts of nodes in a relatively simple and scalable fashion. This has shown its effect on the prevalence of smart homes, security devices, and industrial systems.

On the other hand, Field-Programmable Gate Arrays (FPGAs) are a corner stone of modern day computing. These devices offer much greater flexibility as they can be reprogrammed from the ground up to perform almost any specific task while delivering great speed and efficiency. Because of this, they can be an essential tool in developing as well as prototyping in many industries such as telecommunications, automotive, aerospace, and artificial intelligence.

Usually IoT networks are created with nodes which are microcontrollers communicating together, even if they are from different architecture. In this project, I wanted to combine both aspects of the industry and create an IoT network where the FPGA is a node. In order to achieve this, I wanted to use both an FPGA and a microcontroller as nodes in an IoT network. This would allow for an implementation that uses the hardware power of an FPGA as well as the connectivity offered by a microcontroller.

This project's aim is to demonstrate the flexibility of FPGA design capabilities as well as IoT networks' practicality in an industrial environment where control, transmitting and receiving nodes are all built with different architectures and still be able to communicate with minimum hardware footprint and a non-hierarchical structure.

2 Fundamentals

2.1 Overview

In this section some information are given that are important to understand before going into the body of the work itself. This includes basic concepts, communication protocols and hardware specs.

2.2 Internet of Things (IoT)

The term Internet of Things was first used by Kevin Ashton in 1999[SYHHAB24] during a supply chain management presentation at Procter and Gamble. At that time, this referred mainly to radio-frequency identification (RFID), but since then the term has grown to include many other things. IoT now represents that there are more devices connected to the internet than there are people alive. As of 2022, the number of devices connected to the Internet reached 11.5 billion, growing from only 500 million in 2003[SSS19]. This number is projected to grow to 25 billion by the year 2030[SEK+22].

Before this huge increase in number of connected devices, the usual structure for a wireless devices network used to be hierarchical as seen in figure 2.1. As the number of connected devices increase, this hierarchical structure was not practical any more and was then substituted by a decentralized network. As a result of this change, some of the architectures that were used in these networks had to be redesigned. The most important examples of these changes were the addressing and protocols. In most IoT networks now IPv6 addresses are used instead of the old addresses of IPv4. Where IPv4 allowed around 4.3 billion unique addresses, IPv6 allows $3.4 * 10^{38}$ unique addresses among many other security features[WCW+13]. For the protocols used, there are different ones used depending on the complexity,security and speed requirements.

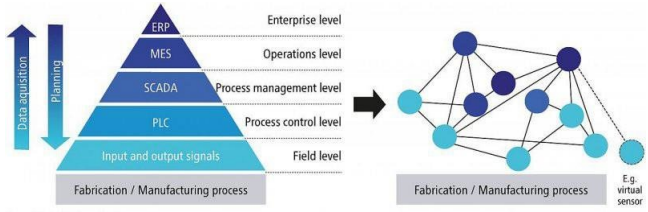

Figure 2.1: Traditional automation pyramid vs IoT network[But19].

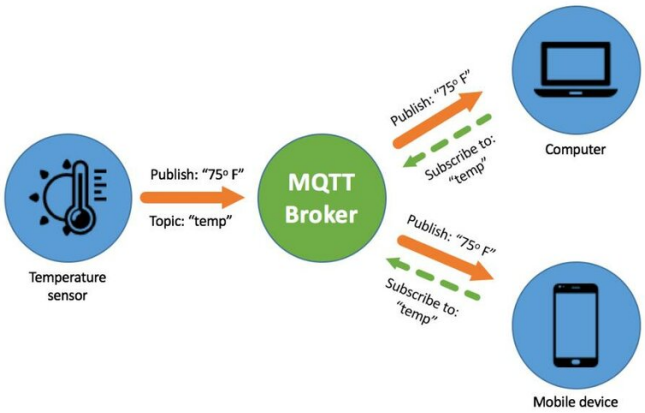

Figure 2.2: Example of MQTT network structure[Sug20]

2.3 MQTT

MQTT (Message Queuing Telemetry Transport) is a communication protocol developed in 1999 by Dr. Andy Stanford-Clark and Arlen Nipper[Tea23]. The MQTT protocol is built on the subscriber/publisher model and is designed to be lightweight and bandwidth efficient and at the same time retain high degree of quality of service. The structure of an MQTT network can be seen in figure2.2 which consists of three main parts :

- Broker
- Publisher
- Subscriber

It is important to note that each node in the network can be a publisher and a subscriber at different times during its program and that there can be multiple brokers as well to divide the load and prevent bottle-necking. The network works by the publisher sending the information to the broker under a specific *topic* (e.g. Temperature) on the other hand each node that has subscribed to the same topic gets forwarded the information by the broker. MQTT topics can also be hierarchical as a branching tree spaced by (/)[Pro23] e.g. Room_reading/Temperature.

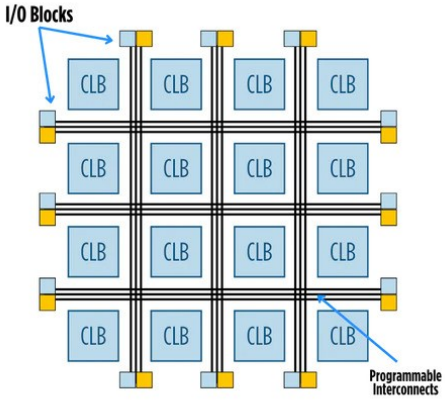

Figure 2.3: FPGA building blocks [Inc]

2.4 FPGA

Field-programmable gate arrays (FPGAs) are reprogrammable computer chips that can be designed to create any specific hardware design. FPGAs are created from a grid of programmable blocks, but unlike Application Specific Integrated Circuits (ASICs) where the wiring connections between the blocks are permanent, in FPGAs these bonds can be made and then undone using specific programming commands and software that changes the wiring between these blocks each time it runs. In Figure2.3, we can see how the Configurable Logic Blocks (CLBs) and IO blocks are connected together using the reprogrammable interconnects. Such functionality allows FPGAs to be used in many different fields. Because they are flexible in their use in comparison to ASICs they are very important during prototyping phases or even for production in low to medium scale production levels[Inc].

FPGAs are programmed using what's called Hardware Description Languages (HDLs). These are similar to programming languages but still differ in that -as the name implies-, they are responsible for defining how these CLBs are configured and interconnected. The most popular of these HDLs are Verilog (including systemVerilog) and Very High-Speed Integrated Circuit Hardware Description Language (VHDL).

2.5 Raspberry Pi Pico W

The Raspberry Pi Pico W is a microcontroller from the Pico family that has the ability to connect over WiFi. It is powered by the RP2040 microcontroller with dual core cortex M0+ processor with up to 133 MHz.

Figure 2.4: Raspberry Pi Pico W board[Fou]

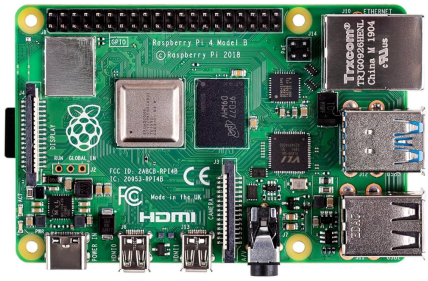

Figure 2.5: Raspberry pi 4 model B board[Ras23]

It also offers 264KB SRAM and 2MB of on board flash memory. The board offers also 26 GPIO pins that enable multiple protocols as I2C, SPI and UART[Fou22]. The Pico W is also programmable in Micropython, CircuitPython as well as C/C++ which makes it very flexible in its usage depending on the complexity of the software the designer wants to realize. As the Pico W is such a small and relatively low power device it is a good example of an IoT node.

2.6 Raspberry Pi 4

The Raspberry Pi 4 is a single-board computer (SBC) designed to be used in various applications. Although it is decently powered with a Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit SoC running at 1.5 GHz along with 8 GB of LPDDR4 RAM, connectivity options such as WiFi, Bluetooth, and gigabit Ethernet and a variety of ports including HDMI, USB 3.0, USB C, and 40 GPIO pins supporting different protocols, it still offers a small footprint of only 85 mm by 56 mm, which is roughly the side of a credit card[Fou19].

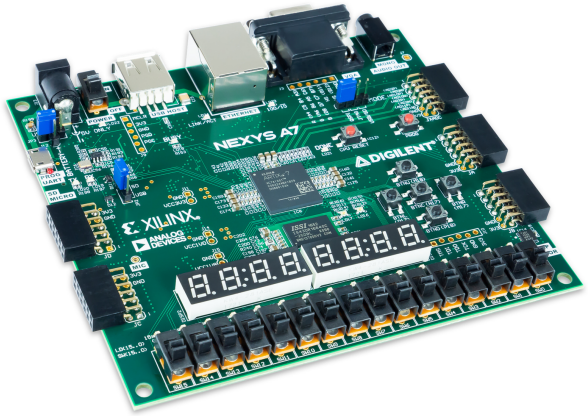

Figure 2.6: Nexys A7-100T board[Inc18]

2.7 Nexys A7-100T

The Nexys A7-100T is an FPGA development board developed by Digilent. It is mainly suggested for learning the basics of FPGA design even though it is also decently capable. It uses the Xilinx Artix-7 FPGA chip, specifically the XC7A100T-1CSG324C model. This chip offers 101,440 logic cells as well as 4,860 Kb of block Ram.[Inc18]

The development board offers also 256 MB of DDR2 SDRAM as well as 16 MB Quad SPI Flash memory and an additional SD slot. It offers also very rich connectivity options including USB, Ethernet and PMOD connectors. The board also offers multiple switches, LEDs, push buttons and an 8 digit 7-segment display for interactions as well as multiple sensors on board.

The board can be programmed using both HDL or Verilog using the Xilinx Vivado Design Suite in order to synthesis, implement and generate the bitstream of the design.

3 Methodology

In order to realize the goals of the project defined in chapter 1, I decided to use the following hardware components:

- Nexys A7-100T FPGA board
- Raspberry Pi Pico w
- Raspberry Pi 4 model B

The FPGA board is used in this case to read the temperature from the internal temperature sensor (ADT7420) which requires an I2C interface in order to communicate the readings. This temperature is then displayed as binary using a series of 8 LEDs and also on the 7 segment display. In this case the FPGA board resembles a machine in an industrial setting that continuously gives updates on its status.

Then, in order to be able to communicate wirelessly, the FPGA board is connected to the Pico W using SPI protocol which will use the Mosquitto MQTT client to publish the received temperature readings through a broker.

On the other end the Raspberry Pi 4 is set up as both a Mosquitto MQTT broker and a subscriber as well where the readings can be viewed through the linux terminal.

An overview of the structure of the project can be seen in Figure 3.1 showing a block diagram of the project

This project was first planned to be an implementation of the MQTT protocol on the FPGA directly using a soft-core MicroBlaze V processor running PetaLinux. This application can eliminate the need for the Pico W but would still need additional work to connect a wireless PMOD connection or to use the Gigabit Ethernet on board. This however, was not possible due to time constraints. Even though this might be more work on the FPGA board itself, this can be seen as a more realistic application of FPGA in the industrial field as no additional hardware is needed as well as it would be a more advanced implementation that uses much more of the FPGA

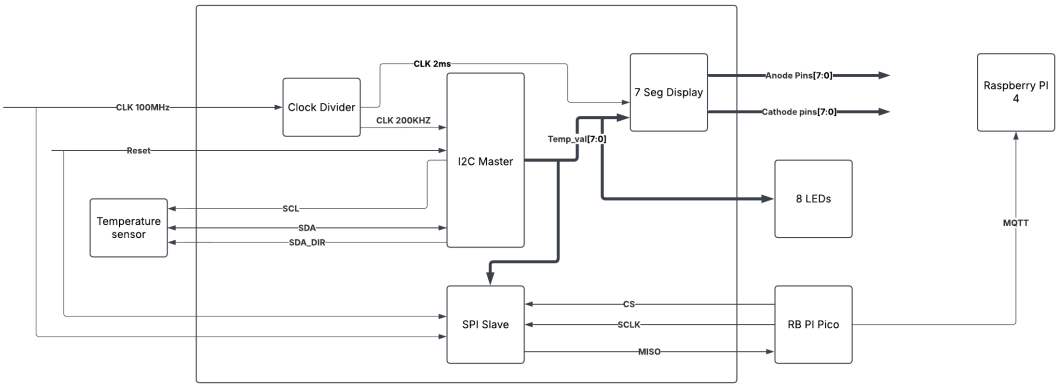

Figure 3.1: Block diagram of the project

capabilities that normal microcontrollers can't replicate.

4 Evaluation

4.1 Overview

In this section the exact blocks of the projects will be explained and insights will be given on why these were built in this way. As both the microcontroller and the SBC are programmed using Python, their code is relatively high level which means it would be simpler and more abstract. As a result, most of this section will be spent going over the FPGA VHDL code as it is much more detailed and important to get the project working.

4.2 FPGA Design

The FPGA design was created as set of components. Each of these components corresponds to a block in the block design in Figure3.1. Then all this is combined into a parent component that connects to the external inputs and outputs. Each of the components created has a corresponding test bench where the functionality is simulated to evaluate whether it is designed correctly or not. In this section I will go over the components in brief detail as well as some of the waveforms of their test benches

4.2.1 Clock Divider

The Nexys A7 board has a 100 MHz oscillator already built in, but for our program we need two different clocks and none of them can be running at that high of a frequency. For the I2C protocol a clock of 200 KHz is required while for the seven segment display a 500 Hz clock is required to give the display a 2 ms of turn on time.

In order to achieve this, the clock divider component is created, taking in the 100 MHz clock as input and providing 2 outputs corresponding to the 2 clocks. To change the value of the clocks, a ticker is created for each of them. The ticker value is incremented every clock cycle until it reaches its

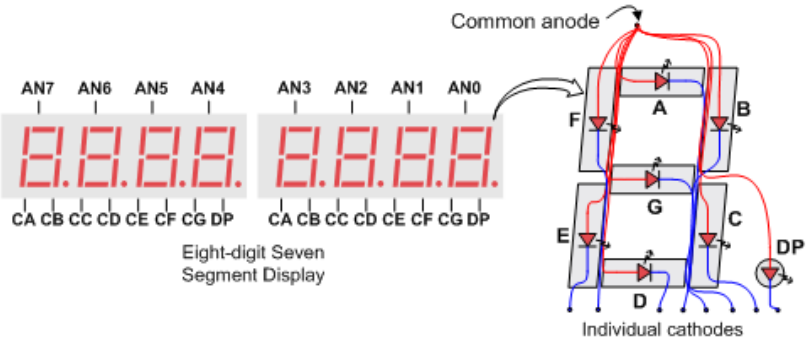

Figure 4.1: Seven segment illustration[Inc18]

maximum value. Once the maximum value is reached, the output clock corresponding to the ticker is switched to the opposite of its current state (e.g. from high to low). The maximum value of the ticker for a clock of frequency $freq$ can be calculated with the equation 4.1

$$Ticker_{max} = \frac{100 * 10^6}{freq} - 1 \tag{4.1}$$

For example, the 200 KHz clock is switched every time its ticker reaches 249. After this, the ticker is reset and starts counting up again.

4.2.2 Seven Segment Display

The seven segment display on the Nexys A7 board is designed in common anode mode where the anode needs to be driven high and the cathode pins need to be driven low, as shown in figure 4.1 The component for the seven-segment display takes in two inputs. The first is the 500 Hz clock generated from the clock divider and the second is the temperature as an 8 bit value got from the temperature sensor (will be explained next). The outputs are the anode and cathode pins which are defined as a bit vector of length 7 and 6 respectively.

First step done is to break down the temperature value into units and tens. This is done through the following lines of code.

```
units_val <= TO_INTEGER(unsigned(temp_val)) mod 10;
tens_val <= TO_INTEGER(unsigned(temp_val))/10;
```

Listing 4.1: Splitting temperature value into units and tens

Once every clock cycle, the anode pin value changes from "11111110" that

enables only the right most digit on the seven segment display which corresponds to the units value, to "11111101" which corresponds to displaying the tens value. After the anode pins are selected, the cathode pins are selected based on a switch case for the number that needs to be shown. The values of these pins are shown in the following code listing.

```
case number_to_show is
  when 0 =>
  cathode_pins <= "0000001";
  when 1 =>
  cathode_pins <= "1001111";
  when 2 =>
  cathode_pins <= "0010010";
  when 3 =>
  cathode_pins <= "0000110";
  when 4 =>
  cathode_pins <= "1001100";
  when 5 =>
  cathode_pins <= "0100100";
  when 6 =>
  cathode_pins <= "0100000";
  when 7 =>
  cathode_pins <= "0001111";
  when 8 =>
  cathode_pins <= "0000000";
  when 9=>
  cathode_pins <= "0001100";
        when others =>
        cathode_pins <= "0001100";
end case;
```

Listing 4.2: Switch case for the cathode pins

A test bench simulation was created for this as can be seen in figure 4.2. Here the display is simulated switching between the values 25, 43 and 78 each staying on for 4 ms. We see how each clock cycle both the cathode and anode pins change between the values for units and tens in order.

4.2.3 I2C Master

The Nexys A7 board has an Analog Device ADT7420 temperature sensor which requires an I2C interface to communicate the temperature readings. For this, this component was created, in order to act as the I2C master in the communication. As a result, this component has two inputs which are the 200 KHz clock and the reset button. It also has two outputs which are the final temperature reading and the SCL line which carries the output I2C clock which will be fed into the sensor. additional two inout lines are needed, one of them is the SDA line carrying the serial bit in both directions

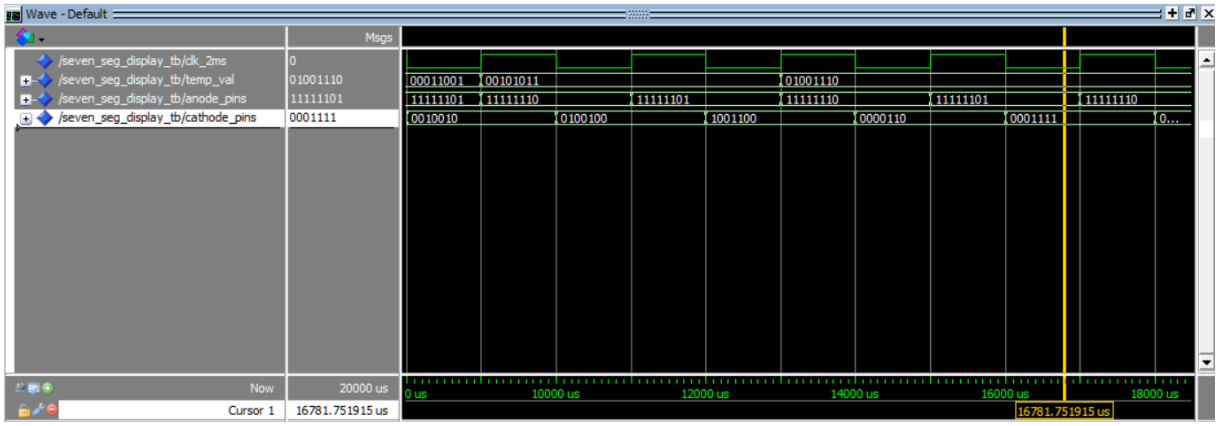

Figure 4.2: Test bench simulation for seven segment display

and the second is the SDA_dir line which is set to 1 in case of sending data and 0 in case of receiving.

It is important to have the output SCL clock slower than the input clock, as the change of SDA line bit needs to happen mid-transition of the SCL clock. This is shown in detail according to the ADT7420 in figure 4.3. This figure shows the transmission happening in a specific sequence. In order to implement this sequence of actions, a VHDL state machine is designed. The sequence of these diagrams can be seen visualized in figure 4.4.

It is also important to note that since the SDA line is shared it is crucial that the master is only using the line when it is actively sending information (e.g. address or acknowledgment). This is why the SDA_dir value is very important as the SDA line is is set to z whenever the the SDA_dir is not 1.

These steps in action can be seen in action in figure 4.5. The sequence starts by sending the device address "1001011" and the read bit "1" on the SDA line. The SDA line is then released as "z" waiting to receive the acknowledgment and the temperature MSB which will be saved in the "thermal_data" variable. The SDA line is then pulled low to send an acknowledgment and is then released to receive the temperature LSB. At the end the SDA line is pulled high to send the NACK and is ready to repeat the cycle again. It is important to note that after receiving the the temperature MSB and LSB, they are not just combined together. The resulting 16-bit vector has flags in bits 2 down to 0 and since the number given is in two's complement, the first bit is a sign bit. As the temperatures we are measuring in this project are always going to be above $0°C$, then we can focus on the bit vector in the range [14:3]. The sensor is able to read also in decimals with resolution of $0.0625°C$, but in order to be able to display it on the seven segment display we can focus on the integer part

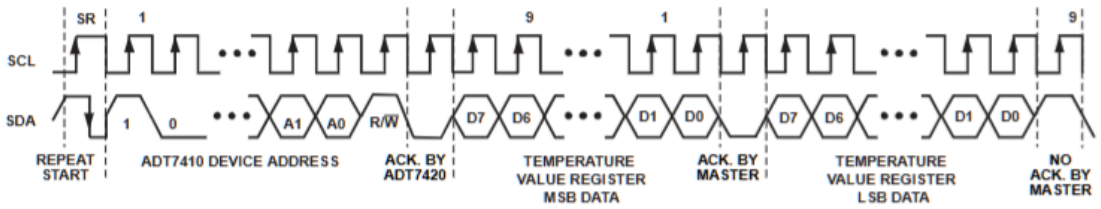

Figure 4.3: Required bit sequence to receive the temperature values[AD24]

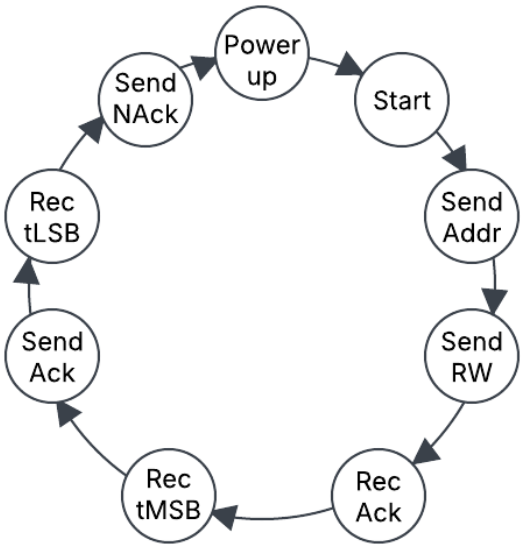

Figure 4.4: VHDL state machine visualized

by removing the last 4 bits of the bit vector. As a result, we can only take the bits [14:7] which is an 8 bit vector.

4.2.4 SPI Slave

The SPI communication protocol relies on a master and at least one slave nodes. The main differences between it and the I2C mentioned before, is that SPI allows communications at much higher speeds than that of SPI and that it is also full duplex which means that it has two separate lines for sending and receiving bits which can be running at the same time unlike I2C which only has one SDA shared line. These lines are Master In Slave Out (MISO), Master Out Slave In (MOSI). Along side these lines, as it allows multiple slaves, SPI also requires a Slave Select (SS) going from the master to the slave(s) which is normally high and is pulled low on the slave that is going to communicated with. All the required lines can be seen in figure 4.6 As the communication is going to be happening between the Pico

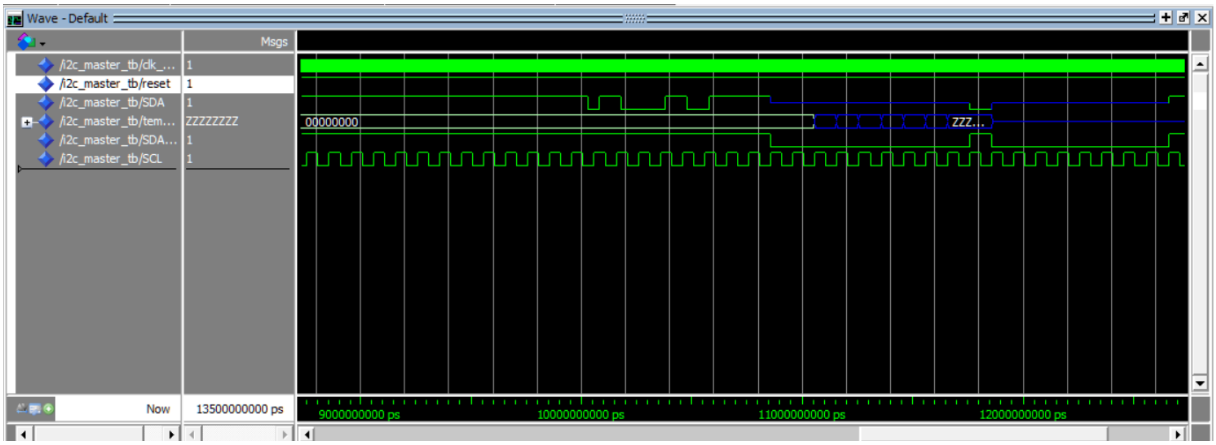

Figure 4.5: Test bench simulation of I2C Master

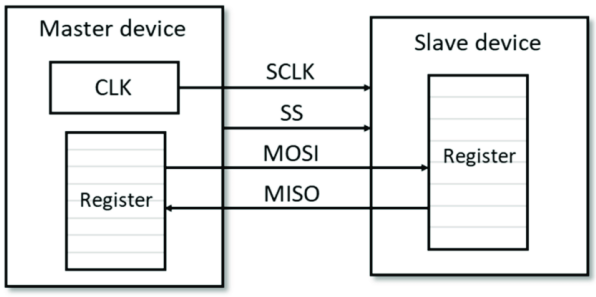

Figure 4.6: SPI block diagram[CCT+20]

W and the FPGA board, a decision had to be made of which one is going to be master and which one is going to be the slave. In reality however, the Pico W only supports being the SPI master, so the FPGA board needs to be the slave. This means that there will be no internal clock generation in the VHDL as it would be an input from the master. As the communication is going to be mainly the FPGA sending over the temperature readings on the MISO line, we can disregard the MOSI line in this implementation, which makes the VHDL code simpler still.

In order to avoid any meta-stable states, an synchronized clock is created which is an internal signal that is set to the value of the clock at its edge and manage all internal variable changes depending on this synced clock. The output waveforms can be seen in figure 4.7 where the value of the data to be sent is changed from "10101010" and "11000101". It is important to notice that between each cycle for the data the SS line must be pulled up to 1 and then pulled down to 0 to initiate the communication again. After this the MISO line starts to serialize the DIN variable which holds the data needed to be transferred.

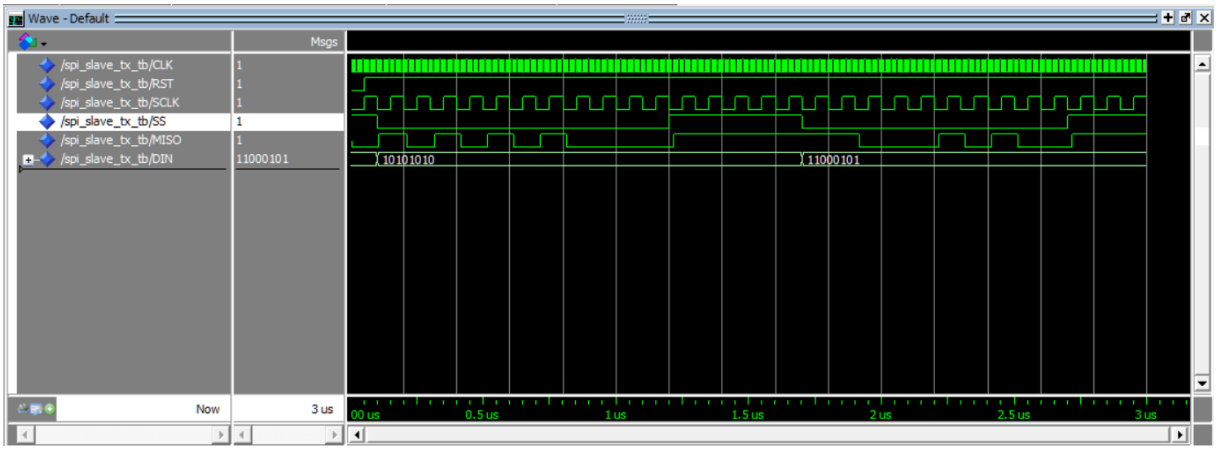

Figure 4.7: Test bench of SPI slave component

4.3 Pico W Design

Since the Pico W only needs to act as a wireless node, building the design with Micropython can be sufficient. The program for this requires mainly three steps :

4.3.1 Setting up SPI interface

The Pico W board allows for 2 SPI interfaces to be connected together. In this application only one (SPI0) is used. This requires the 4 lines mentioned in figure 4.6 which can be connected to GPIO pins 16 to 19 and on the FPGA side they are connected to the PMODA connector as can be seen in figure 4.8. These connections need to be defined also in software. For this, the SPI and Pin functions are imported from the machine Micropython library. As a requirement in the SPI function, the phase and polarity of the SPI signal needs to be defined. SPI has 4 modes available in total as visualized in figure 4.9. In this application I went with polarity mode of 0 and phase of 0 as well. The clock polarity doesn't make a huge change in this case but the phase change can allow a more stable reading of the values as the sampling will be happening on the rising edge of the clock which would coincide with the middle of the change from the FPGA's side. In listing 4.3, the setup for the SPI in micropython is shown in further details. A good thing about using the internal SPI setup of the Pico W is that it handles the pull-up resistors of all the necessary pins without any external pins. Since the data received is in the form of bytes, the function *bytes_to_decimal* is needed in order to transfer easy-to-read numbers to the broker.

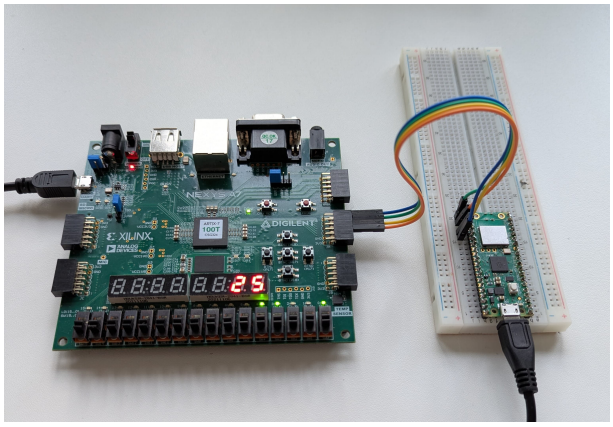

Figure 4.8: SPI interface connection between FPGA and Pico W

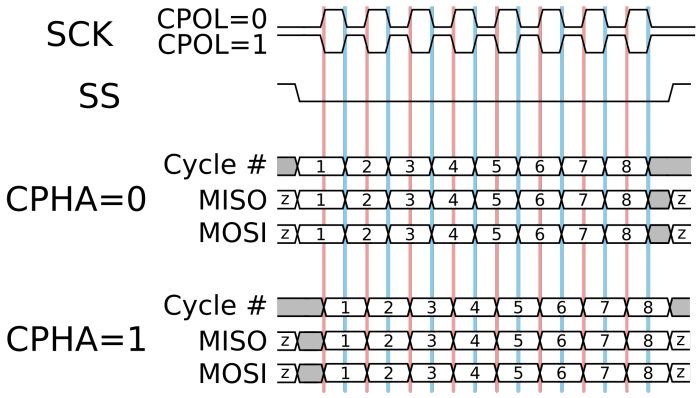

Figure 4.9: SPI modes[WA04]

```
from machine import Pin, SPI
import time

# SPI Configuration
spi = SPI(0, baudrate=1000000, polarity=0, phase=0, sck=Pin(18), mosi=
    Pin(19), miso=Pin(16))
ss = Pin(17, Pin.OUT)  # Chip Select (ss) pin
def receive_data_from_fpga():
    ss.value(0)  # Activate the chip select (active low)
    data = spi.read(1)
    ss.value(1)  # Deactivate the chip select
    return data
def bytes_to_decimal(data):
    # Convert bytes to a list of decimal values
    return [int(byte) for byte in data]
```

Listing 4.3: SPI setup in Python

4.3.2 MQTT setup

The setup of the MQTT is also assisted by an existing Micropython library, namely the *simple.py* library. As shown in listing 4.4, the MQTT Client function is imported which takes in two arguments which are the name of the client and the IP address of the broker which in this case is the IP address of the Pi 4 board. Another function needed is the $connectToWifi$ which takes in the SSID and SSID password from the constants file. After the setup of the MQTT client is finished, we are ready to define the publish function. Basically, this is the same as the default publish function but only with a try,except statement in case of any errors found and that it would print the sent value and topic for debugging in console.

```
import network,constants
from simple import MQTTClient
from connectToWifiLib import connectToWifi
#setting up the client
def mqttConnect():
    client = MQTTClient(constants.mqttClient, constants.mqttBroker,
   keepalive=100)
    client.connect()
    print('MQTT connected')
    return client
def makeConnection():
    connectToWifi(constants.SSID,constants.SSID_password)
    return mqttConnect()
#defining publish function
def publish(topic,value,client):
        try:
            client.publish(topic, value)
            print(f"sent value {value} to topic {topic}")
        except OSError:
            print('Error: MQTT connection failed')
```

Listing 4.4: MQTT setup

4.3.3 Main file

Because the previous steps were made in separate files to act as local libraries, the main file is kept relatively clean and organized. As can be seen in listing 4.5, the code starts by importing the functions defined in the previous sections. After this the topic is defined and then the main loop starts. The loop starts by receiving the data from the SPI line, transforming it into decimal and then publishing it to the predefined topic.

```
import time
from SPI_master import receive_data_from_fpga,bytes_to_decimal,
   send_data_to_fpga
from mqtt_send import makeConnection,publish
```

```
#creating MQTT client
client=makeConnection()
mqttTopic = "FPGA/Temp"
while True:
    received_data = receive_data_from_fpga()
    # Convert the received bytes to decimal values
    decimal_data = bytes_to_decimal(received_data)
    print("Received Data (Decimal):", (decimal_data))
    publish(mqttTopic,str(decimal_data),client)
    time.sleep(1)
```

Listing 4.5: main file code

4.4 Raspberry Pi Design

On the Raspberry Pi's side the process is fairly simple. The process is only setting up the MQTT Mosquitto client. To install the client, the package installer needs to be updated through the first line in listing 4.6. In line 2 the Mosquitto client itself is installed. After this, the Mosquitto clients is initiated using the third line in the listing.

```
    sudo apt update && sudo apt upgrade
    sudo apt install -y mosquitto mosquitto-clients
    mosquitto -v
    mosquitto_sub -t "FPGA/Temp"
```

Listing 4.6: Linux commands to set up MQTT

Once the subscription is done, the terminal gets updated automatically after every publishing done by the Pico W.

4.5 Results

Once the FPGA board gets connected to the Vivado software, it is ready to be flashed. This requires the synthesis of the project which is translating the VHDL code into a netlist of logic gates and connections between them. This is followed by the implementation, which routes this netlist of logic gates into the FPGA board itself. Finally comes the bitstream generation which converts everything to a binary file that is loaded onto the FPGA chip itself in order to program it. Once this is done, the FPGA board is connected to the Pico board as shown in figure 4.8 and to the computer via USB in order to view the terminal. The Raspberry pi doesn't need to connected to anything in order to work, but in order to view the results, it

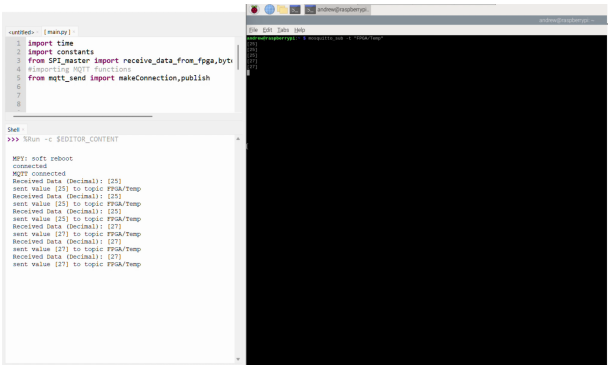

Figure 4.10: From left to right, terminal of Pico W and Raspberry Pi

will be connected to the same PC for power and to view the terminal as well. It is important that all devices are connected to same WiFi network, it doesn't have to have an active internet connection but just so that the devices can locate each other using their IP addresses. In figure 4.10 we can see the terminals of each device and we can see the information being passed correctly.

5 Summary

5.1 Conclusion

This project was able to successfully demonstrate the integration of FPGA into an IoT network using a microcontroller. This highlights the potential that the combination of these technologies can offer in industrial and personal applications as well. As this project implemented multiple communication protocols, namely SPI, I2C and MQTT, the project shows how across different protocols, architectures and programming languages, communication can still be reliable and quick. The FPGA's flexibility in handling sensor data, the microcontroller's role in wireless communication, and the Raspberry Pi's function as a broker collectively emphasized the practicality of such a system in real-world scenarios.

This project could theoretically have been completed by connecting the temperature sensor pins directly to the Pico W and using the built in I2C interface pins on it without the use of any VHDL code or design. While this indeed would be a lot simpler, but this would not have achieved the goals set out for this project. This project's aim was to utilize the FPGA as much as possible and utilize the microcontroller as little as possible. This is why building both the SPI and I2C interfaces from scratch in VHDL is a better fit for this project rather than building none or just using the same I2C interface for both the temperature sensor and the microcontroller communication as well.

This ties back to the goal set for the initial project where no microcontroller was needed at all. As with a Linux image booted on the MicroBlaze-V architecture, allows the FPGA to achieve much more complex and different tasks that would normally not be achievable using just microcontrollers.

5.2 Future plans

Even though this project worked to the extent of the goals set in the beginning, there are still improvements that can be done. Some of these improvements are:

1. Including an online dashboard that can monitor and control the temperature sensor readings.
2. Another node in the IoT network that can react to changes in the readings (e.g. safety mechanism in case the temperature gets too high or too low).
3. Implement a real full duplex system on the SPI interface with both MISO and MOSI exchanging data at the same time.
4. Designing of a PCB board containing the FPGA chip, Pico W and all the other needed I/Os in order to reduce the footprint of the system.

These improvements are ways that this project can better simulate a real-life application which can better show FPGAs can play role in the implementation of IoT networks.

Bibliography

[AD24] Inc. Analog Devices. *ADT7420: ±0.2°C Accurate, 16-Bit Digital I²C Temperature Sensor*, 2024. Accessed: 2025-03-09.

[But19] Florian Butollo. Butollo (2019) data, artificial intelligence and industrial internet platforms. 06 2019.

[CCT+20] Shih Lun Chen, Tsun-Kuang Chi, Min-Chun Tuan, Chiung-An Chen, Liang-Hung Wang, Wei-Yuan Chiang, Ming-Yi Lin, and Patricia Angela Abu. A novel low-power synchronous preamble data line chip design for oscillator control interface. *Electronics*, 9:1509, 09 2020.

[Fou] Raspberry Pi Foundation. Raspberry pi pico microcontroller image. `https://www.raspberrypi.com/documentation/microcontrollers/images/pico-1s.png?hash=37f90d0137e81859630d4f0943b6f3d5`. Accessed: 2025-03-09.

[Fou19] Raspberry Pi Foundation. Raspberry pi 4 model b technical specifications, 2019. Accessed: 2025-03-09.

[Fou22] Raspberry Pi Foundation. Raspberry pi pico w technical specifications, 2022. Accessed: 2025-03-09.

[Inc] C&T Solution Inc. Fpga architecture diagram. `https://cdn.shopify.com/s/files/1/0028/7509/7153/files/1_2_2bb5d72f-6dd0-452d-8705-2b5931c26131_480x480.png?v=1710223274`. Accessed: 2025-03-09.

[Inc18] Digilent Inc. Nexys a7-100t fpga board technical specifications, 2018. Accessed: 2025-03-09.

[Pro23] Tizian Prokosch. Essential guide to mqtt topics and wildcards. *Cedalo*, 1:1–1, 2023.

[Ras23] Raspberry Pi Foundation. Raspberry pi os (formerly raspbian) image for raspberry pi 4. `https://www.raspberrypi.com/products/raspberry-pi-4-model-b/`, 2023. Accessed: 2023-10-01.

[SEK+22] Prashant Singh, Zeinab Elmi, Vamshi Krishna Meriga, Junayed Pasha, and Maxim A. Dulebenets. Internet of things for sustainable railway transportation: Past, present, and future. *Cleaner Logistics and Supply Chain*, 4:100065, 2022.

[SSS19] Neha Sharma, Madhavi Shamkuwar, and Inderjit Singh. The history, present and future with iot. In Vijender Kumar Solanki, Valentina E. Balas, Manju Khari, and Raghvendra Kumar, editors, *Internet of Things and Big Data Analytics for Smart Generation*, volume 154 of *Intelligent Systems Reference Library*, pages 27–51. Springer International Publishing, Cham, 2019.

[Sug20] Kalaivanan Sugumar. Mqtt -a lightweight communication protocol relative study, 05 2020.

[SYHHAB24] Jameel Shehu Yalli, Mohd Hilmi Hasan, and Aisha Abubakar Badawi. Internet of things (iot): Origins, embedded technologies, smart applications, and its growth in the last decade. *IEEE Access*, 12:91357–91382, 2024.

[Tea23] HiveMQ Team. Mqtt essentials part 1: Introducing mqtt, 2023. Accessed: 2025-03-09.

[WA04] Wikipedia-Autoren. Serial peripheral interface, October 2004.

[WCW+13] Peng Wu, Yong Cui, Jianping Wu, Jiangchuan Liu, and Chris Metz. Transition from ipv4 to ipv6: A state-of-the-art survey. *IEEE Communications Surveys Tutorials*, 15(3):1407–1424, 2013.

List of Figures

2.1 Traditional automation pyramid vs IoT network[But19]. . 3
2.2 Example of MQTT network structure[Sug20] 3
2.3 FPGA building blocks [Inc] 4
2.4 Raspberry Pi Pico W board[Fou] 5
2.5 Raspberry pi 4 model B board[Ras23] 5
2.6 Nexys A7-100T board[Inc18] 6

3.1 Block diagram of the project 8

4.1 Seven segment illustration[Inc18] 10
4.2 Test bench simulation for seven segment display 12
4.3 Required bit sequence to receive the temperature values[AD24] 13
4.4 VHDL state machine visualized 13
4.5 Test bench simulation of I2C Master 14
4.6 SPI block diagram[CCT+20] 14
4.7 Test bench of SPI slave component 15
4.8 SPI interface connection between FPGA and Pico W . . . 16
4.9 SPI modes[WA04] 16
4.10 From left to right, terminal of Pico W and Raspberry Pi . 19

Affidavit

I Andrew Antwan Mikheal Fahmy Zakhary herewith declare that I have composed the present paper and work by myself and without use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The paper and work in the same or similar form has not been submitted to any examination body and has not been published. This paper was not yet, even in part, used in another examination or as a course performance. .
Lippstadt, March 16, 2025

Andrew Zakhary